

Introduction to Decision Control Structures & Loops

Definition

By default, a C program executes statements **one after another, from top to bottom** — this is called **sequential execution**. But real programs need to **make decisions** and **repeat actions**, just like we do in everyday life.

Decision Control Structures allow a program to **choose** which block of code to run, based on whether a condition is **true** or **false**. This is also called **Branching**, because the program's flow "branches" into different paths.

Why Do We Need Decision Making?

Think about real life: *"If it is raining, take an umbrella, otherwise don't."* This is exactly how decision control works in programming — the computer checks a **condition** and then decides **what to do next**.

Real Life

"If age ≥ 18, allow voting, otherwise don't."

In C Program

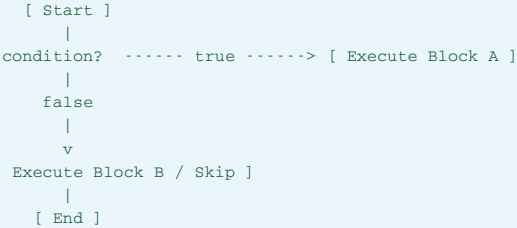
```
if (age >= 18) { printf("Eligible"); }
```

Two Main Categories

Category	Purpose	Statements Covered
Decision Making / Branching	Choose one path out of many, based on a condition	Simple <code>if</code> , <code>if...else</code> , Nested <code>if...else</code> , <code>else...if</code> Ladder, <code>switch</code>
Looping (Iteration)	Repeat a block of statements multiple times	<code>for</code> , <code>while</code> , <code>do...while</code> , <code>break</code> , <code>continue</code> , <code>goto</code>

In this note, we focus on the first category: **Decision Making and Branching Statements**.

The Idea of "Branching"



The program checks the **condition** first. Depending on whether it is **true** or **false**, it follows a **different branch (path)** of execution — this is why these statements are called **branching statements**.

Note

- A condition in C is any expression that evaluates to **true (non-zero)** or **false (0)** — usually written using relational (>, <, ==) or logical (&&, ||) operators.
- Decision control structures always use **curly braces { }** to group multiple statements into one block.
- The four branching forms — simple `if`, `if...else`, nested `if...else`, and `else...if` ladder — are all built from the **same basic if keyword**, just arranged differently.

Fun Tip

Sequential = "no choice" | Branching = "computer picks a path"

Simple if Statement

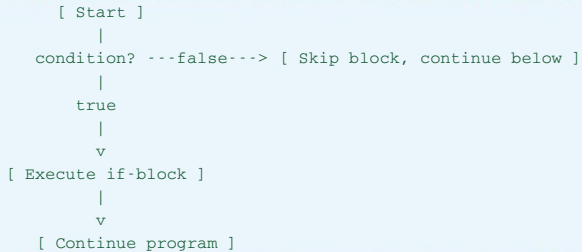
Definition

The **simple if statement** is the most basic decision-making statement in C. It tests a **condition**, and if the condition is **true**, the block of code inside the `if` is executed. If the condition is **false**, that block is **simply skipped**.

Syntax

```
if (condition) {  
    // statements to execute if condition is true  
}
```

Flow of Control



How It Works

- The condition inside () is evaluated first.
- If it is **true (non-zero)**, the statements inside { } run.
- If it is **false (0)**, the block is **ignored completely**, and control moves to the next line after the `if` block.
- There is **no "else"** here — if the condition fails, nothing special happens.

Example Program

```
// Program to check if a number is positive
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num > 0) {
        printf("%d is a positive number\n", num);
    }

    printf("Program finished.\n");

    return 0;
}
```

► Output (if input = 7)

```
Enter a number: 7
7 is a positive number
Program finished.
```

► Output (if input = -3)

```
Enter a number: -3
Program finished.
```

Note

- If the `if` block has **only one statement**, the curly braces { } are optional — but using them is a **good habit** for readability.
- The condition must always be inside **round brackets ()** — this is compulsory.
- Simple `if` is best used when you only need to do something **if a condition is true**, with no alternative action needed.

* Fun Tip

Simple `if` = "Do this only if true – otherwise, do nothing."

if...else Statement

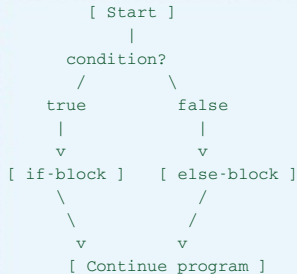
Definition

The **if...else** statement lets the program choose **between two paths**. If the condition is **true**, the if block runs. If it is **false**, the else block runs instead. **Exactly one** of the two blocks will always execute.

Syntax

```
if (condition) {  
    // statements if condition is true  
} else {  
    // statements if condition is false  
}
```

Flow of Control



How It Works

- The condition is evaluated **once**.
- If **true** → only the if block executes, and the else block is skipped.
- If **false** → only the else block executes, and the if block is skipped.
- Unlike simple if, there is **always some action** — either one branch or the other.

Example Program

```
// Program to check if a number is even or odd  
#include <stdio.h>  
  
int main() {  
    int num;  
  
    printf("Enter a number: ");  
    scanf("%d", &num);  
  
    if (num % 2 == 0) {  
        printf("%d is Even\n", num);  
    } else {  
        printf("%d is Odd\n", num);  
    }  
  
    return 0;  
}
```

Output (if input = 10)

```
Enter a number: 10  
10 is Even
```

Output (if input = 7)

```
Enter a number: 7  
7 is Odd
```

Note

- The else block can **never exist alone** — it must always be attached to an if.
- There is **no condition** written with else — it simply means "in all other cases."
- Use if...else when you have exactly **two possible outcomes** (true / false).

Fun Tip

if = "Plan A" | **else** = "Plan B" — one of them always runs!

Nested if...else Statement

Definition

A **nested if...else** means placing **one if...else statement inside another if or else block**. It is used when a decision depends on **more than one condition**, checked **one after another**, in a hierarchy.

Syntax

```
if (condition1) {  
    if (condition2) {  
        // runs if condition1 AND condition2 are true  
    } else {  
        // runs if condition1 is true but condition2 is false  
    }  
} else {  
    // runs if condition1 is false  
}
```

How It Works

- The **outer** condition is checked **first**.
- Only if the outer condition is **true**, the program moves on to check the **inner (nested)** condition.
- If the outer condition is **false**, the inner if...else is **never checked** at all.
- This is useful when the second condition **only makes sense** after the first one is satisfied.

Example Program

```
// Program to find the largest of three numbers using nested if-else  
#include <stdio.h>  
  
int main() {  
    int a = 10, b = 25, c = 18;  
  
    if (a > b) {  
        if (a > c) {  
            printf("a is the largest: %d\n", a);  
        } else {  
            printf("c is the largest: %d\n", c);  
        }  
    } else {  
        if (b > c) {  
            printf("b is the largest: %d\n", b);  
        } else {  
            printf("c is the largest: %d\n", c);  
        }  
    }  
  
    return 0;  
}
```

Output

```
b is the largest: 25
```

Step-by-Step: Tracing the Example

1. Outer check: $a > b \rightarrow 10 > 25 \rightarrow$ **false**, so go to the else part.
2. Inner check: $b > c \rightarrow 25 > 18 \rightarrow$ **true**.
3. So "b is the largest: 25" is printed.

Note

- Nesting can go **multiple levels deep**, but too much nesting makes code **hard to read** — 2 to 3 levels is usually a practical limit.
- **Proper indentation** is very important in nested if statements, so you can easily match each if with its corresponding else.
- Nested if...else is a good choice when conditions are **dependent on each other** (i.e., the inner check only matters after the outer check passes).

Fun Tip

Nested if = "if" inside an "if" — like boxes inside boxes 

else...if Ladder

Definition

The **else...if ladder** (also written as **if...else if...else**) is used when you need to check **many conditions one after another**, and execute the block for the **first condition that turns out true**. It looks like a "ladder" of conditions stacked one below another.

Syntax

```
if (condition1) {
    // runs if condition1 is true
} else if (condition2) {
    // runs if condition1 is false AND condition2 is true
} else if (condition3) {
    // runs if condition1, condition2 are false AND condition3 is true
} else {
    // runs if ALL conditions above are false
}
```

Flow of Control

```
condition1? --true--> [ Block 1 ] --> Continue
|false
v
condition2? --true--> [ Block 2 ] --> Continue
|false
v
condition3? --true--> [ Block 3 ] --> Continue
|false
v
[ else Block (default) ] --> Continue
```

How It Works

- Conditions are checked **from top to bottom, in order**.
- As soon as **one condition is true**, its block runs, and **all remaining checks are skipped** — control jumps straight out of the ladder.
- If **none** of the conditions are true, the final `else` block (if present) runs as the **default case**.
- The final `else` is **optional** — you can have a ladder with no default case.

Example Program

```
// Program to assign a grade based on marks
#include <stdio.h>

int main() {
    int marks = 72;

    if (marks >= 90) {
        printf("Grade: A+\n");
    } else if (marks >= 75) {
        printf("Grade: A\n");
    } else if (marks >= 60) {
        printf("Grade: B\n");
    } else if (marks >= 40) {
        printf("Grade: C\n");
    } else {
        printf("Grade: Fail\n");
    }

    return 0;
}
```

Output

Grade: B

Step-by-Step: Tracing the Example (marks = 72)

1. `marks >= 90` → `72 >= 90` → false, move to next check.
2. `marks >= 75` → `72 >= 75` → false, move to next check.
3. `marks >= 60` → `72 >= 60` → **true** → prints "Grade: B" and exits the ladder (remaining checks are skipped).

Note

- `else...if` ladder is best when checking **many different values of the same variable** — e.g. grading, menu choices.
- Only the block for the **first true condition** executes — the rest are automatically skipped, even if they'd also be true.
- Compare with nested `if...else`: a ladder checks **independent, sequential** conditions, while nested `if` checks conditions that are **dependent on each other**.

Fun Tip

Think of it as going down a **ladder** 🪜 — you stop at the first step that fits!